

Job Skill Extraction Using NLP

DAY 1 – Understand the Problem

Date: 21 Aug 2026

Introduction: -

This project focuses on automatically extracting relevant skills from job descriptions using Natural Language Processing (NLP). The system will process job description text and identify the job role and required skills.

1. Problem Statement

Job descriptions contain important information about the required skills, experience, and qualifications of a candidate. However, job descriptions are generally available as unstructured text, making manual identification of required skills time-consuming and difficult. The objective of this project is to develop an NLP-based system that can automatically identify and extract relevant skills from job descriptions.

2. Business Objective

- Reduce manual effort in recruitment.
- Quickly identify required skills from job descriptions.
- Help recruiters screen job requirements efficiently.
- Support better job and candidate matching.
- Identify frequently demanded skills in the job market.
- Improve the overall efficiency of the recruitment process.

3. Technical Objective

The technical objective of this project is to develop an NLP-based system that accepts job descriptions as input and extracts the job role, relevant technical and non-technical skills. The system will use Python and NLP techniques to clean, process and analyse job description text and produce a structured list of extracted skills.

4. Input

Input: - The input to the system will be a job description in text format.

Example: - we require a Data Scientist with python, Pandas, NumPy, Scikit-Learn, SQL and AWS experience.

5. Expected Output

Job Role: - Data Scientist

Skills: -

- Python
- Pandas
- NumPy
- Scikit-learn
- SQL
- AWS

Expected Output: -

The system should identify the job role and extract the relevant skills from the given job description in a structured format.

6. Technologies: -

Technology	Purpose
Python	Programming
Google Colab Notebook	Development Environment
Pandas	Data handling
NumPy	Numerical operation
NLPK	NLP Processing
spaCy	NLP and entity extraction
Scikit-learn	Machine Learning
Transformers	Advanced NLP
Matplotlib	Visualization
Seaborn	Visualization
Power BI	Data Visualization
Flask	API deployment

7. Success Criteria: -

The Project will be considered Successful if: -

1. The system accepts a job description as input
2. The system identifies the relevant job role.
3. The system extracts relevant skills from the job description.
4. The extracted information is presented in a structured format.
5. The system can process multiple job descriptions efficiently.

❖ Questions for understanding

1. What is a job description?

A Job Description (JD) is a document or text that provides information about a particular job role. It generally describes the job responsibilities, required skills, qualifications, experience, education, and other requirements expected from a candidate.

Example: - A company is looking for a Data Scientist with experience in Python, Pandas, NumPy, Scikit-learn, SQL, and AWS.

In this project, the job description is the main source of information from which relevant skills will be extracted.

2. What is a skill?

A skill is a specific ability, knowledge, or expertise that a person needs to perform a particular job or task effectively.

Skills can be broadly classified into:

Technical Skills

- Python
- SQL
- Machine Learning
- Pandas
- AWS
- Power BI

Soft Skills

- Communication
- Leadership
- Teamwork
- Problem Solving

For this project, the main focus is on identifying relevant skills mentioned in job descriptions.

3. Why is Automatic Skill Extraction Useful?

Automatic Skill Extraction is useful because job descriptions contain a large amount of unstructured text. Manually reading every job description and identifying skills can be time-consuming. An automated NLP-based system can identify relevant skills quickly and consistently.

Benefits:

- Reduces manual effort.
- Saves time during recruitment.
- Helps identify required skills quickly.
- Makes it easier to compare different job descriptions.
- Helps analyse frequently demanded skills.
- Supports better job and candidate matching.
- Can process a large number of job descriptions efficiently.

Therefore, automatic skill extraction can improve the efficiency of recruitment and job analysis processes.

4. Difference Between Keyword Extraction, Entity Extraction, Classification and Information Extraction

These are different NLP techniques used to process and understand text.

Technique	Meaning	Example
Keyword Extraction	Identifies important words or phrases from text	Python, SQL, Machine Learning
Entity Extraction	Identifies specific entities such as people, organizations, locations, etc.	Microsoft → Organization, Pune → Location
Classification	Assigns text or information to predefined categories	Python → Programming, AWS → Cloud
Information Extraction	Extracts useful structured information from unstructured text	Job Title, Skills, Experience, Location

▪ Keyword Extraction

Keyword Extraction is the process of identifying important words or phrases that represent the main information in a text.

Example:

Input: - We are looking for a Data Analyst with experience in Python, SQL and Power BI.

Output:

- Python
- SQL
- Power BI

- **Entity Extraction**

Entity Extraction is the process of identifying specific entities mentioned in text, such as people, organizations, locations, dates, or other predefined entities.

Example:

Input: - Tanuja worked at Infosys in Pune.

Output:

- Organization → Infosys
- Location → Pune
- Person → Tanuja

- **Classification**

Classification is the process of assigning information to predefined categories.

For our project, extracted skills can be classified into different categories.

Example:

Python → Programming

SQL → Database

AWS → Cloud

Power BI → Data Visualization

Scikit-learn → Machine Learning

Information Extraction

Information Extraction (IE) is the process of automatically extracting useful information from unstructured text and converting it into a structured format.

Example:

Input: - We require a Data Scientist with 3 years of experience in Python, SQL and AWS.

Output:

- Job Role → Data Scientist
- Experience → 3 Years
- Skills → Python, SQL, AWS

5. Define Project Input and Output

Project Input

The main input of this project is a job description in unstructured text format.

Example:

We require a Data Scientist with Python,

Pandas, NumPy, Scikit-learn, SQL and AWS experience.

The DAY 1 task itself gives this type of job description as the project example.

Project Output

The system should process the job description and produce a structured output containing the job role and extracted skills.

Example:

Role: Data Scientist

Skills:

- Python
- Pandas
- NumPy
- Scikit-learn
- SQL
- AWS

This role-and-skills output format is also shown in the task document.

Date: 22 Aug 2026

DAY 2 – Dataset Collection

This dataset contains 2525 job records with details such as job title, company, location, job description, skills, experience, education, and salary. It is created to analyse job descriptions and prepare the data for NLP-based skill extraction. The dataset also includes some missing and duplicate records for practicing data cleaning.

Columns	Description
Job_id	Unique identifier assigned to each job
Job_title	Title r role of the job
Company	Company associates with the job
Company_name	Name of the hiring company
Company_id	Unique identifier of the company
Location	Location where the job is available
Job_description	Detailed description of the job and required skills
Keywords	Important keywords or skills related to the job
Experience	Required years or level of experience
Education	Required educational qualification
Salary	Salary or salary range offered
Job_time	Type or duration of the job

Dataset Purpose

- The main purpose of this dataset is to:
- Understand the structure of job-related data.
- Analyze job titles, locations, companies and descriptions.
- Identify and clean inconsistent or duplicate records.
- Preprocess job descriptions for NLP.
- Prepare the cleaned text for automatic skill extraction in the later stage of the project.

#Dataset Basic Analysis

1. Dataset Shape

Purpose: To check the total number of records and columns present in the dataset

```
df.shape
```

```
(2525, 12)
```

2. Dataset Columns

Purpose: To Identify all the columns available in the dataset.

```
print(df.columns)
```

```
Index(['job_id', 'job_title', 'company', 'company_name', 'company_id',  
      'location', 'job_description', 'keywords', 'experience', 'education',  
      'salary', 'job-time'],  
      dtype='object')
```

3. First Five Records

Purpose: To get a quick overview of the data and understand the structure of the records.

```
df.head()
```

	job_id	job_title	company	company_name	company_id	location
0	JOB101812	BI Analyst	InsightWorks	InsightWorks	COM007	Chennai, Tamil Nadu
1	JOB102215	ML Engineer	BluePeak Digital	BluePeak Digital	COM011	Mumbai, Maharashtra
2	JOB100557	Financial Analyst	ByteCraft Solutions	ByteCraft Solutions	COM008	Nagpur, Maharashtra
3	JOB101269	Data Analyst	FinEdge Technologies	FinEdge Technologies	COM006	Pune, Maharashtra
4	JOB100757	product analyst	Quantix Analytics	Quantix Analytics	COM009	Nagpur, Maharashtra

4. Missing values Check

Purpose: To identify missing values present in each column of the dataset.


```
#Missing value
print(df.isnull().sum())
```

```
job_id          0
job_title       0
company         0
company_name    12
company_id      0
location        33
job_description  25
keywords        30
experience      20
education       18
salary          22
job-time        15
dtype: int64
```

5. Duplicate Records Check

Purpose: To identify duplicate records in the dataset.

```
#Duplicate value
duplicate_count = df.duplicated().sum()
print("Number of Duplicate Records:",duplicate_count)
```

```
Number of Duplicate Records: 25
```

6. Job Description Length Check

Purpose: To understand the length of job description and check the text data before NLP processing

```
#Check TEXT length
df["description_length"] = df["job_description"].str.len()
print(df[["job_description","description_length"]].head())
```

	job_description	description_length
0	We are looking for a BI Analyst to join Insigh...	364.0
1	We are looking for a ML Engineer to join BlueP...	357.0
2	We are looking for a Financial Analyst to join...	359.0
3	We are looking for a Data Analyst to join FinE...	374.0
4	We are looking for a Product Analyst to join Q...	358.0

```
#Check Statistical Job-description Length
print("Average Job Description Length:",df["description_length"].mean())
print("Minimum Job Description Length:",df["description_length"].min())
print("Maximum Job Description Length:",df["description_length"].max())
```

```
Average Job Description Length: 361.5224
Minimum Job Description Length: 334.0
Maximum Job Description Length: 392.0
```

Date: 24 Aug 2026

Day 3 – Exploratory Data Analysis (EDA)

Objective:

To perform EDA on the job dataset and understand the distribution, structure, and quality of the available job data.

1.Number of Jobs

Purpose: to find the total number of job records available in the dataset.

```
#Day 3- Exploratory Data Analysis(EDA)
#Number of job
print("Number of Job:",len(df))
```

Number of Job: 2525

2.Number of Unique Job Titles

Purpose: to identify the number of different job roles present in the dataset.

```
#Unique Job Titles
print("Unique Job Titles:",df["job_title"].nunique())
```

Unique Job Titles: 30

3.Most Common Job Roles

Purpose: To identify the roles that occur most frequently in the dataset.

```
# Most common job Roles
common_job_roles = df["job_title"].value_counts().head(10)
print("Most Common Job Roles:")
print(common_job_roles)
```

```
Most Common Job Roles:
job_title
AI Engineer      182
Software Engineer 178
Product Analyst  176
Data Scientist   174
Python Developer 173
Financial Analyst 171
MIS Executive    169
BI Analyst       168
Junior Data Analyst 166
ML Engineer      164
Name: count, dtype: int64
```

4.Job by Location

Purpose: To understand how job opportunities are distributed across different locations.

```
df["location"].value_counts().head(10)
```

Output:

location	count
Ahmedabad, Gujarat	239
Pune, Maharashtra	217
Delhi NCR	217
Gurugram, Haryana	215
Mumbai, Maharashtra	213
Nagpur, Maharashtra	212
Bengaluru, Karnataka	210
Remote	201
Chennai, Tamil Nadu	194
Noida, Uttar Pradesh	190

5.Average Description Length

Purpose: To calculate the average length of job description.

```
#Average description Length
average_description_length = df["description_length"].mean()
print("Average Description Length:",average_description_length)
```

Average Description Length: 357.9429702970297

6.Minimum Description Length

Purpose: To identify the shortest job description in the dataset.

```
#Minimum Description Length
min_description_length = df["description_length"].min()
print("Minimum Description Length:",min_description_length)
```

Minimum Description Length: 0

7.Maximum Description Length

Purpose: TO identify longest job description in the dataset.

```
#maximum Description Length
max_description_length = df["description_length"].max()
print("Maximum Description Length:",max_description_length)
```

Maximum Description Length: 392

8.Missing values Analysis

Purpose: To identify missing values in each column of the dataset.

```
#Missing Value
missing_summary = pd.DataFrame({
    "Missing Values": df.isnull().sum(),
    "Missing Percentage": (df.isnull().sum() / len(df)) * 100
})
print(missing_summary)
```

	Missing Values	Missing Percentage
job_id	0	0.000000
job_title	0	0.000000
company	0	0.000000
company_name	12	0.475248
company_id	0	0.000000
location	33	1.306931
job_description	25	0.990099
keywords	30	1.188119
experience	20	0.792079
education	18	0.712871
salary	22	0.871287
job-time	15	0.594059
description_length	0	0.000000

9.Duplicate Records Analysis

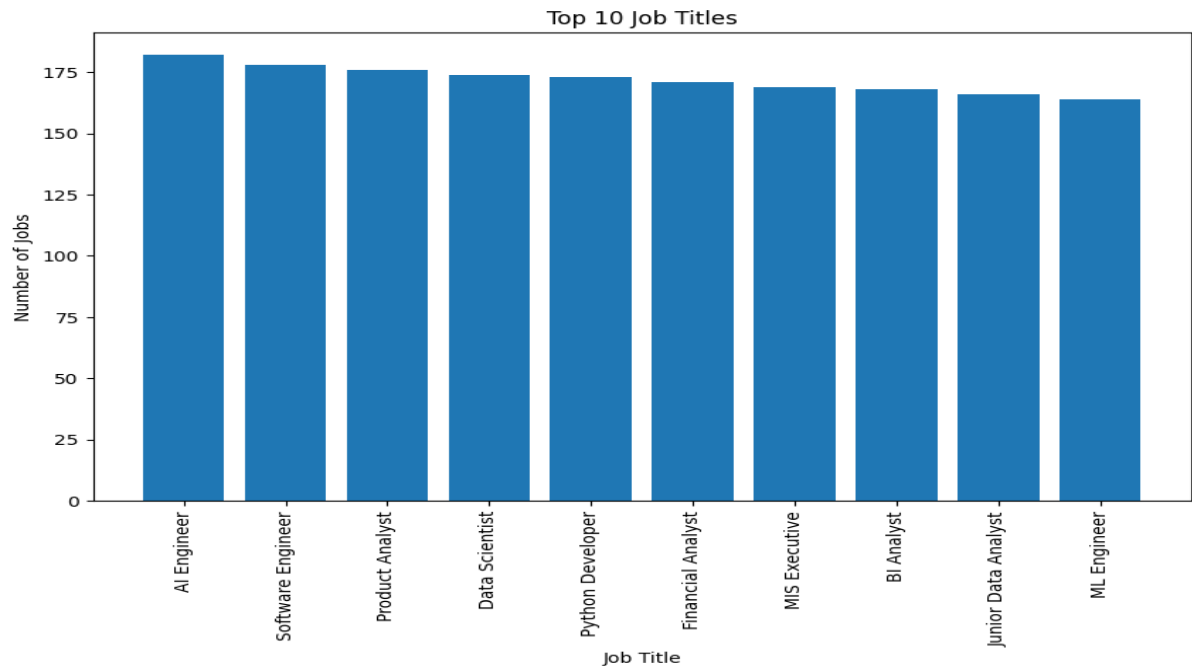
Purpose: To identify duplicate records present in the dataset

```
#Duplicate Records
duplicate_count = df.duplicated().sum()
print("Number of Duplicate Records:",duplicate_count)
```

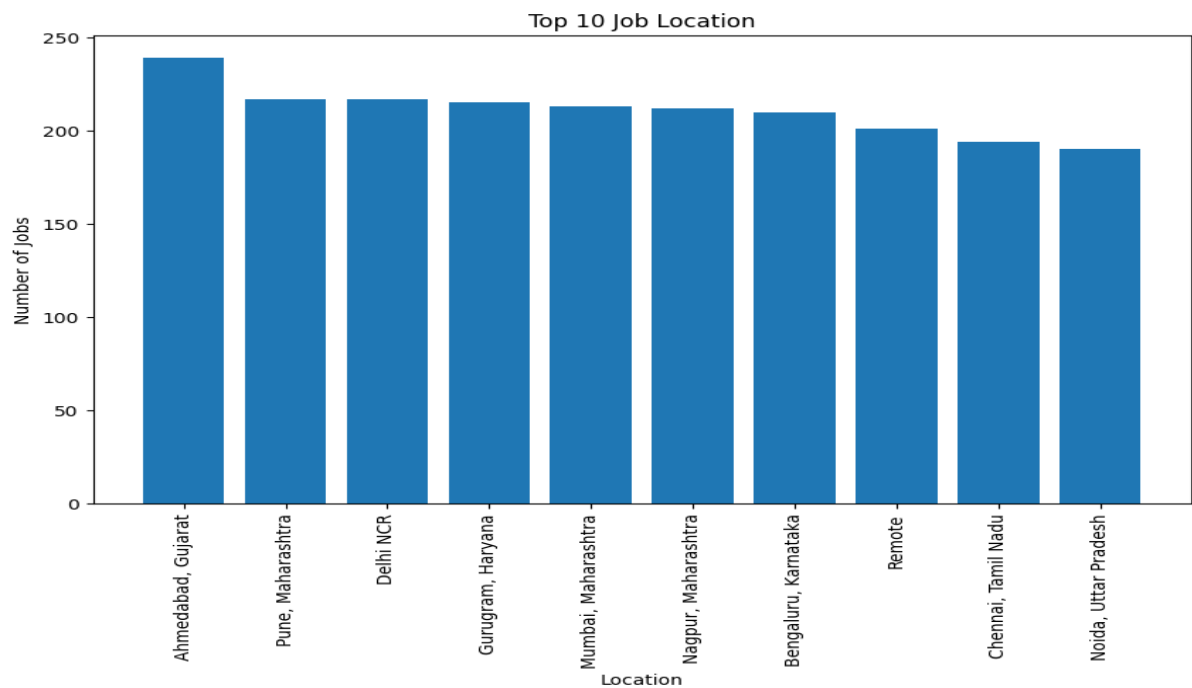
Number of Duplicate Records: 25

Day 3 – Visualizations

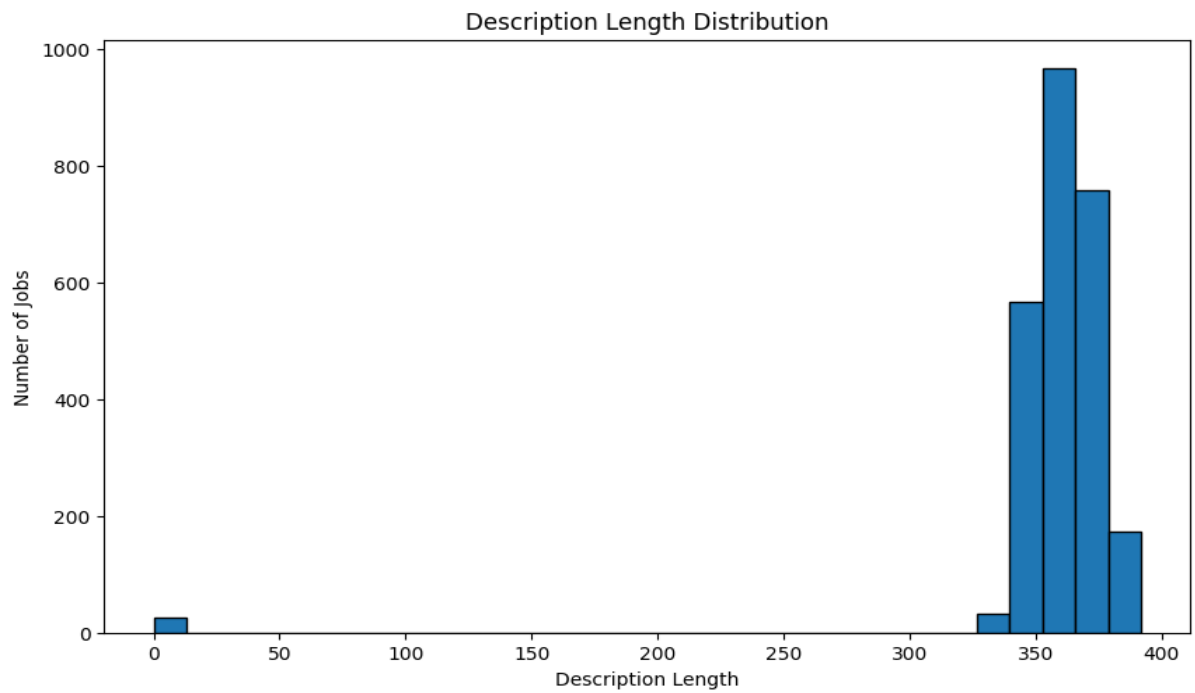
1.Top Job Titles



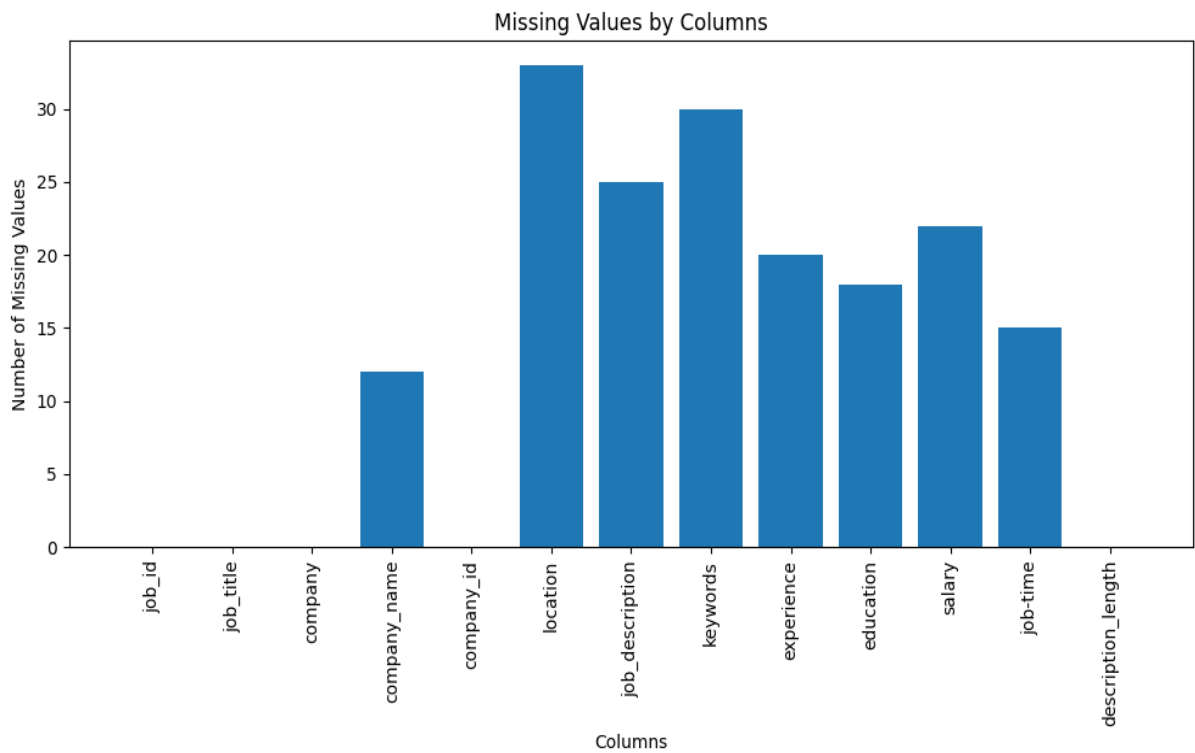
2.Job By Location



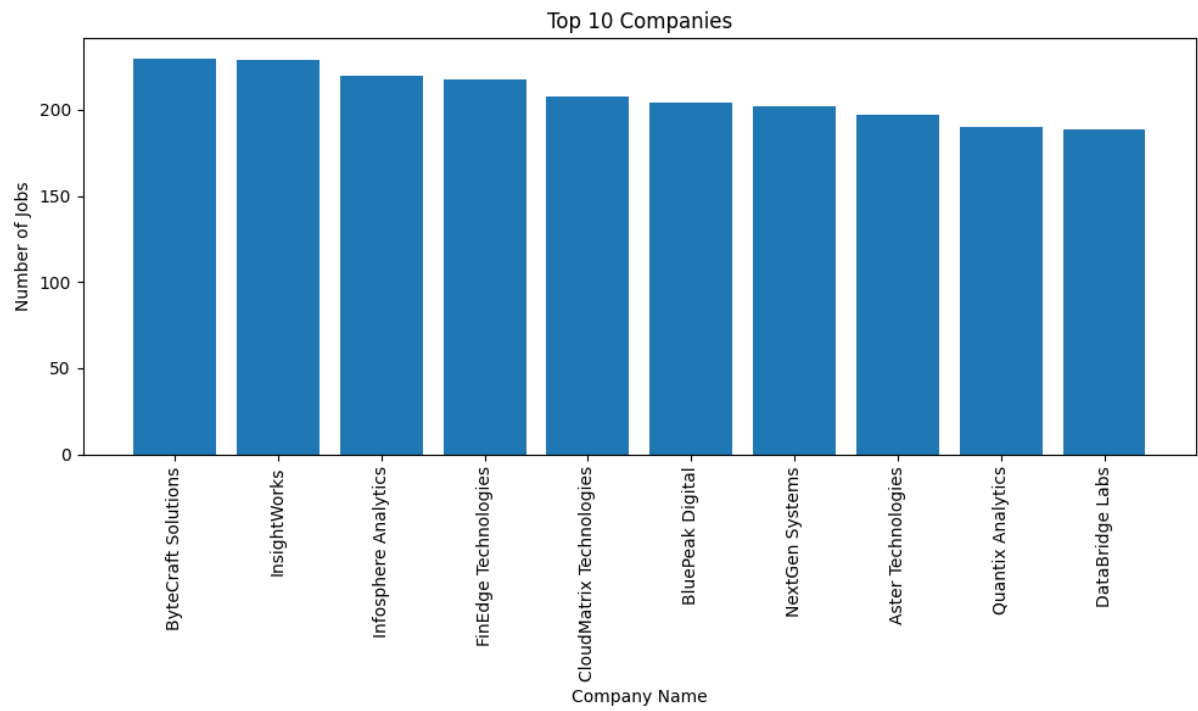
3.Description Length Distribution



4.Missing values



5.Top companies



Date: 25 Aug 2026

Day 4- Data Cleaning

Objectives:

To clean the job description by removing unwanted content and preparing the text for further NLP processing

1.Data Cleaning Requirements

The job description was cleaned by handling the following unwanted elements:

- HTML tags
- URLs
- Email addresses
- Phone number
- Extra spaces
- Duplicate text
- Unnecessary symbols

2.Create the Text Cleaning Function

Purpose: A cleaning function was created to remove unwanted elements from the job description and standardize the text.

3.Apply Cleaning Function

Purpose: The cleaning function was applied to the job_description column to create a new cleaned text column.

```
df["cleaned_job_description"] = df["job_description"].apply(clean_text)
```

4.Compare Original and Cleaned Text

Purpose: To compare the original job descriptions with the cleaned descriptions and verify the cleaning process

```
df[["job_description","cleaned_job_description"]].head(10)
```


Output:

...	job_description	cleaned_job_description
0	We are looking for a BI Analyst to join Insigh...	we are looking for a bi analyst to join insigh...
1	We are looking for a ML Engineer to join BlueP...	we are looking for a ml engineer to join bluep...
2	We are looking for a Financial Analyst to join...	we are looking for a financial analyst to join...
3	We are looking for a Data Analyst to join FinE...	we are looking for a data analyst to join fine...
4	We are looking for a Product Analyst to join Q...	we are looking for a product analyst to join q...
5	We are looking for a Software Engineer to join...	we are looking for a software engineer to join...
6	We are looking for a Financial Analyst to join...	we are looking for a financial analyst to join...
7	We are looking for a Data Scientist to join As...	we are looking for a data scientist to join as...
8	We are looking for a Product Analyst to join I...	we are looking for a product analyst to join i...
9	We are looking for a MIS Executive to join Ins...	we are looking for a mis executive to join ins...

5.Validate the Cleaning

Purpose: To check whether unwanted URLs, Email, addresses, and HTML tags are still present after cleaning

```
#cleaning Summary
print("Cleaning Summary")
print("Total Jobs:",len(df))
print("original Description column:", "job_description")
print("Cleaned description column:", "clean_job_description")
print("Remaning URLs:",remaining_urls)
print("Remaining Emails:",remaining_emails)
print("Empty Cleaned Descriptions:",empty_description_count)
print("Average Cleaned Job Description Length:",df["cleaned_job_descripti
```

```
Cleaning Summary
Total Jobs: 2525
original Description column: job_description
Cleaned description column: clean_job_description
Remaning URLs: 0
Remaining Emails: 0
Empty Cleaned Descriptions: 0
Average Cleaned Job Description Length: 346.34376237623763
```

Date: 25 AUG 2026

DAY 5 – Create the Skill Taxonomy

Objectives:

To create a master skill dictionary for the job dataset, categorize the skills into different groups, and define common aliases for the skills.

1.Master Skill Dictionary

A master skill dictionary was created to maintain a standard list of commonly used technical skills. The skills include Python, SQL, Java, C++, Power BI, Tableau, Excel, AWS, Azure, GCP, TensorFlow, PyTorch, Pandas, NumPy, Scikit-learn, Spark, Hadoop, Docker, and Git.

Output: - Total Skills: 19

```
['python', 'SQL', 'Java', 'C++', 'Power BI', 'Tableau', 'Excel', 'AWS', 'Azure', 'GCP', 'TensorFlow', 'PyTorch', 'Pandas', 'NumPy', 'Scikit-learn', 'Spark', 'Hadoop', 'Docker', 'Git']
```

```
... Total skills: 19
['python', 'SQL', 'Java', 'C++', 'Power BI', 'Tableau', 'Excel', 'AWS', 'A
```

2.Skill Categories

The skills were grouped into different categories such as Programming, Database, Data Analytics, machine Learning, and Cloud

Output: - Total Skill Categories: 20

```
{'Python': 'Programming', 'Java': 'Programming', 'C++': 'Programming', 'SQL': 'Database', 'Power BI': 'Data Analytics', 'Tableau': 'Data Analytics', 'Excel': 'Data Analytics', 'Scikit-learn': 'Machine Learning', 'TensorFlow': 'Machine Learning', 'PyTorch': 'Machine Learning', 'AWS': 'Cloud', 'Azure': 'Cloud', 'GCP': 'Cloud', 'Pandas': 'Data Analytics', 'NumPy': 'Data Analytics', 'Spark': 'Data Analytics', 'Hadoop': 'Data Analytics', 'Docker': 'Cloud', 'Kubernetes': 'Cloud', 'Git': 'Programming'}
```

```
Total Skill Categories: 20
{'Python': 'Programming', 'Java': 'Programming', 'C++': 'Programming', 'SQ
```

3.Skill Aliases

Common variations or aliases of skills were identified and mapped to their standard skill names. For example, python3 is mapped to Python, MS excel to Excel, Postgres and PostgreSQL to PostgreSQL, powerbi to Power BI, and scikit learn to Scikit-learn.

Output: - {'python3': 'Python', 'MS excel': 'Excel', 'Postgres': 'PostgreSQL', 'PostgreSQL': 'PostgreSQL', 'powerbi': 'Power BI', 'scikit learn': 'Scikit-learn'}

...	skill	category	aliases
0	Python	Programming	python3
1	SQL	Database	sql
2	Power BI	Data Analytics	powerbi, power-bi
3	PostgreSQL	Database	postgres, postgresql
4	Excel	Data Analytics	ms excel
5	Scikit-learn	Machine Learning	scikit learn

4.Create the Skill Taxonomy

The skill, category, and alias information were combined into a structured taxonomy.

Output: -

...	skill	category	aliases
0	Python	Programming	python3
1	SQL	Database	sql
2	Java	Programming	
3	C++	Programming	
4	Power BI	Data Analytics	powerbi, power-bi
5	Tableau	Data Analytics	
6	Excel	Data Analytics	ms excel
7	AWS	Cloud	
8	Azure	Cloud	
9	GCP	Cloud	
10	TensorFlow	Machine Learning	
11	PyTorch	Machine Learning	
12	Pandas	Data Analytics	
13	NumPy	Data Analytics	
14	Scikit-learn	Machine Learning	scikit learn
15	Spark	Data Analytics	
16	Hadoop	Data Analytics	
17	Docker	Cloud	

5.Validate the Skill Taxonomy

Check Total Skills

```
print("Total Skills in Taxonomy:", len(skill_taxonomy_df))
```

Output: - Total Skills in Taxonomy: 21

Check Duplicate Skills

```
print("Duplicate Skills:", skill_taxonomy_df["skill"].duplicated().sum())
```

Output: - Duplicate Skills: 0

Date: 26 AUG 2026

DAY 6 – NLP Text Preprocessing

1.Objective

It is to preprocess job descriptions using Natural Language Processing (NLP) techniques. The main purpose is to clean and prepare the text so that it can be used for further skill extraction.

2.Dataset Used

The job dataset contains job related information such as job title, company, location, job description, experience, education, salary, and job type. The job_description columns are mainly used for NLP preprocessing.

3.NLP Preprocessing Step

Step 1 -Tokenization: -

Tokenization is the process of splitting text into individual words or tokens.

Example:

Original Text: - We are looking for candidates with strong experience in Python and SQL.

Tokenized Text:

We, are, looking, for, candidates, with, strong, experience, in, Python, and, SQL.

```
#Sample JOB Description
text = df["cleaned_job_description"].iloc[0]
print(text)
```

```
we are looking for a bi analyst to join insightworks the candidate should
```

```
#Tokenization
from nltk.tokenize import word_tokenize
tokens = word_tokenize(text)
print(tokens)
```

```
['we', 'are', 'looking', 'for', 'a', 'bi', 'analyst', 'to', 'join', 'insig
```

Step 2 – Lowercasing: -

In this step, all text is converted into lowercase to maintain consistency.

Example: - Python, SQL, Power BI

Output: - python, sql, power bi

```
#Lowercasing
lower_text = text.lower()
print(lower_text)
```

```
we are looking for a bi analyst to join insightworks the candidate should
```

Step 3 – Stopword Removal

Stopword removal removes common words that are not very useful for further text analysis.

Example:

Before: - Looking for candidates with strong experience in Python and SQL.

After stopwords removal: - Looking candidates strong experience Python SQL

```
#Stopword Removal
```

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))
filtered_tokens = [word for word in tokens if word.lower() not in stop_wo
print(filtered_tokens)
```

```
['looking', 'bi', 'analyst', 'join', 'insightworks', 'candidate', 'experie
```

Step 4 – Lemmatization

Lemmatization converts words into their meaningful base or dictionary form.

Example:

candidates → candidate

running → run

```
#Lemmatization

from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
lemmatized_tokens = [lemmatizer.lemmatize(word) for word in filtered_tokens]
print(lemmatized_tokens)

['looking', 'bi', 'analyst', 'join', 'insightworks', 'candidate', 'experience']
```

Step 5 – Stemming Comparison

Stemming reduces words to their root form by removing word endings.

In this step, the results of stemming and lemmatization are compared to understand the difference between both techniques.

Example: - Looking for candidates with strong experience in Python and SQL.

Output: - Look for candidate with strong experience in Python and SQL.

```
#Stemming
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
stemmed_tokens = [stemmer.stem(word) for word in filtered_tokens]
print(stemmed_tokens)

['look', 'bi', 'analyst', 'join', 'insightwork', 'candid', 'experi', 'table']
```

Step 6 – Sentence Segmentation

Sentence segmentation divides a job description into separate sentences.

Example:

Original: - We are looking for a Data Analyst. The candidate should have experience with Python and SQL.

After segmentation:

We are looking for a Data Analyst.

The candidate should have experience with Python and SQL.



#Stemming vs Lemmatization Comparison

```
comparison = list(zip(filtered_tokens, lemmatized_tokens, stemmed_tokens))
for original, stemmed, lemmatized in comparison[:20]:
    print(f"Original: {original}, Stemmed: {stemmed}, Lemmatized: {lemmatized}")
```



```
Original: looking, Stemmed: looking, Lemmatized: look
Original: bi, Stemmed: bi, Lemmatized: bi
Original: analyst, Stemmed: analyst, Lemmatized: analyst
Original: join, Stemmed: join, Lemmatized: join
Original: insightworks, Stemmed: insightworks, Lemmatized: insightwork
Original: candidate, Stemmed: candidate, Lemmatized: candid
Original: experience, Stemmed: experience, Lemmatized: experi
Original: tableau, Stemmed: tableau, Lemmatized: tableau
Original: sql, Stemmed: sql, Lemmatized: sql
Original: excel, Stemmed: excel, Lemmatized: excel
Original: business, Stemmed: business, Lemmatized: busi
Original: intelligence, Stemmed: intelligence, Lemmatized: intellig
Original: responsibilities, Stemmed: responsibility, Lemmatized: respons
Original: include, Stemmed: include, Lemmatized: includ
Original: data, Stemmed: data, Lemmatized: data
Original: analysis, Stemmed: analysis, Lemmatized: analysi
Original: reporting, Stemmed: reporting, Lemmatized: report
Original: dashboard, Stemmed: dashboard, Lemmatized: dashboard
Original: creation, Stemmed: creation, Lemmatized: creation
Original: data, Stemmed: data, Lemmatized: data
```


Date: 26 AUG 2026

DAY 7 – Rule-Based Skill Extractor

Objective

It is to create a simple rule-based skill extractor that identifies technical skills from job descriptions using a predefined skill dictionary.

1.Create Skill Dictionary: -

```
skills= [ "python", "SQL", "Java", "C++", "Power BI", "Tableau", "Excel", "AWS", "Azure",  
"GCP", "TensorFlow"]
```

```
print("Total Skills:",len(skills))
```

```
print(skills)
```

Output: - Total Skills: 11

```
['python', 'SQL', 'Java', 'C++', 'Power BI', 'Tableau', 'Excel', 'AWS', 'Azure', 'GCP', 'TensorFlow']
```

2. Extraction Logic

The extraction process follows these steps:

- Take the job description as input.
- Convert the text to lowercase.
- Search for each skill from the skill dictionary.
- If the skill is present, add it to the result.
- Return the list of extracted skills.

3. Example

Input Job Description: - We are looking for a candidate with experience in Python, SQL and AWS.

Extracted Skills: - Python, SQL, AWS

4. Result

The rule-based extractor successfully identifies predefined technical skills from job descriptions.

This approach provides a simple way to extract relevant skills from job postings.

...

	job_title	cleaned_job_description	extracted_skills
0	BI Analyst	we are looking for a bi analyst to join insigh...	[SQL, Tableau, Excel]
1	ML Engineer	we are looking for a ml engineer to join bluep...	[SQL, Java]
2	Financial Analyst	we are looking for a financial analyst to join...	[SQL, Power BI, Excel]
3	Data Analyst	we are looking for a data analyst to join fine...	[SQL, Power BI, Excel]
4	product analyst	we are looking for a product analyst to join q...	[SQL, Power BI, Excel]
5	Software Engineer	we are looking for a software engineer to join...	[SQL, Power BI, Excel]
6	Financial Analyst	we are looking for a financial analyst to join...	[python, SQL, AWS]
7	Data Scientist	we are looking for a data scientist to join as...	[python, SQL]
8	Product Analyst	we are looking for a product analyst to join i...	[python]
9	MIS Executive	we are looking for a mis executive to join ins...	[python, SQL, AWS]

Date 27 Aug 2026

DAY 8- Improve Extraction Using Regex and Phrase Matching

Objective

It is to improve the skill extraction process by using Regex-based matching and phrase matching. This helps in identifying different variations of skills and multi-word technical skills from job descriptions.

1. Regex-Based Skill Matching

Regex patterns were used to identify different variations of technical skills from job descriptions.

2. Multi-Word Skill Matching

Regex patterns were also created for multi-word technical skills.

The skills considered include:

- Machine Learning
- Deep Learning
- Natural Language Processing
- Power BI
- Data Science

Regex Pattern Used:

1. "Python": r"\bpython(?:\s*3)?\b",
2. "Machine Learning": r"\bmachine\s+learning\b",
3. "Deep Learning": r"\bdeep\s+learning\b",
4. "Natural Language Processing": r"\bnatural\s+language\s+processing\b",
5. "Power BI": r"\bpower\s*bi\b",
6. "Data Science": r"\bdata\s+science\b"

3. Regex Skill Extraction

A function was created to search the job description using the predefined Regex patterns. When a matching skill is found, it is added to the extracted skills list.

Result

The Regex-based extractor was tested on sample job descriptions and was then applied to the dataset.

Output:

...	job_title	cleaned_job_description	regex_extracted_skills
0	BI Analyst	we are looking for a bi analyst to join insigh...	[]
1	ML Engineer	we are looking for a ml engineer to join bluep...	[]
2	Financial Analyst	we are looking for a financial analyst to join...	[Power BI]
3	Data Analyst	we are looking for a data analyst to join fine...	[Power BI]
4	product analyst	we are looking for a product analyst to join q...	[Power BI]
5	Software Engineer	we are looking for a software engineer to join...	[Power BI]
6	Financial Analyst	we are looking for a financial analyst to join...	[Python]
7	Data Scientist	we are looking for a data scientist to join as...	[Python]
8	Product Analyst	we are looking for a product analyst to join i...	[Python, Machine Learning]
9	MIS Executive	we are looking for a mis executive to join ins...	[Python]

4. Phrase Matching Using spaCy

spaCy PhraseMatcher was used to identify predefined multi-word skills and skill phrases from job descriptions.

The phrases include:

- Machine Learning
- Deep Learning
- Natural Language Processing
- Power BI
- Data Science
- Python
- Python 3
- Python3

5. spaCy Skill Extraction

The PhraseMatcher was applied to job descriptions to identify matching skill phrases.

Result

The spaCy phrase matcher successfully identifies predefined skill phrases from the job descriptions.

Output:

Input:

We need a candidate with Python 3, Machine Learning,
Deep Learning and Power BI experience.

Extracted Skills:

['Python', 'Python 3', 'Deep Learning', 'Power BI', 'Machine Learning']

6. Final Result

The improved skill extraction approach was applied using both Regex matching and spaCy PhraseMatcher.

The extracted skills were stored in the dataset for further analysis.

Output:

	job_title	cleaned_job_description	spacy_extracted_skills
0	BI Analyst	we are looking for a bi analyst to join insigh...	[]
1	ML Engineer	we are looking for a ml engineer to join bluep...	[]
2	Financial Analyst	we are looking for a financial analyst to join...	[power bi]
3	Data Analyst	we are looking for a data analyst to join fine...	[power bi]
4	product analyst	we are looking for a product analyst to join q...	[power bi]
5	Software Engineer	we are looking for a software engineer to join...	[power bi]
6	Financial Analyst	we are looking for a financial analyst to join...	[python]
7	Data Scientist	we are looking for a data scientist to join as...	[python]
8	Product Analyst	we are looking for a product analyst to join i...	[python, machine learning]
9	MIS Executive	we are looking for a mis executive to join ins...	[python]

Date: 28 AUG 2026

Day 9 – TF-IDF Analysis

1. Objective

The objective of Day 9 is to use TF-IDF to identify important terms from job descriptions and compare the identified keywords with the predefined dictionary skills.

2. TF-IDF Vectorization

TF-IDF (Term Frequency-Inverse Document Frequency) was used to identify important terms from the cleaned job descriptions.

3. Parameters Used

max_features = 1000

stop_words = "english"

4. TF-IDF Matrix

TF-IDF Matrix Shape: (2525, 86)

```
#Create TF-IDF Vectorizer
```

```
vectorizer = TfidfVectorizer(max_features=1000, stop_words="english")
```

```
#Apply TF-IDF
```

```
x = vectorizer.fit_transform(df["cleaned_job_description"])  
print("TF-IDF Matrix Shape:", x.shape)
```

```
TF-IDF Matrix Shape: (2525, 85)
```

5. Important TF-IDF Keywords

The highest-scoring terms were identified based on their average TF-IDF scores.

```
... Top 20 TF-IDF Keywords:
      term      score
21      data  0.232851
31    experience 0.196019
12      business 0.115503
75        sql  0.101671
20    dashboard 0.098009
18  collaboration 0.098009
14      candidate 0.098009
15    candidates 0.098009
57    preferred 0.098009
65 responsibilities 0.098009
84        years 0.098009
72      solving 0.098009
62  qualification 0.098009
41        join 0.098009
58      problem 0.098009
64    reporting 0.098009
46      looking 0.098009
35      include 0.098009
19      creation 0.098009
16      cleaning 0.098009
```

6. Dictionary Skills vs TF-IDF Keywords

The TF-IDF keywords were compared with the predefined skill dictionary to identify which dictionary skills were also considered important terms by TF-IDF.

Dictionary Skills Found in TF-IDF:

Common Skills Found: {'python', 'tableau', 'sql', 'numpy', 'aws', 'pandas', 'excel', 'tensorflow', 'spark'}

Comparison Count:

Number of Dictionary Skills Found: 9

```
#Dictionary Skills vs TF-IDF Keywords Compare
dictionary_skills_lower = {skill.lower() for skill in dictionary_skills}
tfidf_keywords_lower = set(tfidf_keywords["term"].str.lower()) #TF-IDF Ke
common_skills = (dictionary_skills_lower.intersection(tfidf_keywords_lowe
print("Dictionary Skills Found in TF-IDF:")
print("Common Skills Found:", common_skills)
```

```
Dictionary Skills Found in TF-IDF:
Common Skills Found: {'numpy', 'python', 'tensorflow', 'excel', 'pandas',
```

```
#Comparison Count
print(
    "Number of Dictionary Skills Found:", len(common_skills)
)
```

```
Number of Dictionary Skills Found: 9
```

Does TF-IDF identify useful skills automatically?

TF-IDF can identify important terms automatically, including some useful technical skills. However, it can also identify general job-related terms. Therefore, TF-IDF is useful for discovering potential skills but may need to be combined with a skill dictionary or other NLP methods for accurate skill extraction.

Date: 28 AUG 2026

DAY 10 – N- Gram Analysis

1. Objective

The objective of Day 10 is to perform N-Gram analysis on job descriptions to identify single-word and multi-word terms that can represent useful skills.

2. N-Gram Analysis

N-Gram analysis was performed to generate Unigrams, Bigrams, and Trigrams from the cleaned job descriptions.

3. N-Gram Vectorizer

```
vectorizer_ngram = TfidfVectorizer(ngram_range=(1,3), stop_words="english")
```

TF-IDF N-Gram Matrix Shape: (2525, 446)

```
#Create N-Gram Vectorizer
vectorizer_ngram = TfidfVectorizer(ngram_range=(1,3), stop_words="english")
```

```
# Create TF-IDF N-Gram Matrix
x_ngram = vectorizer_ngram.fit_transform(df["cleaned_job_description"])
print("TF-IDF N-Gram Matrix Shape:", x_ngram.shape)
```

TF-IDF N-Gram Matrix Shape: (2525, 445)

5. TF-IDF Vectorizer Parameters

`ngram_range = (1, 3)`

`stop_words = "english"`

4. N-Gram Types

Unigram: A single word.

Example: - Python, SQL

Bigram: A combination of two words.

Examples: - Machine Learning, Power BI, Data Science

Trigram: A combination of three words.

Example: - Natural Language Processing

5. Unigram Results

Unigrams were generated to identify important single-word terms from the job descriptions.

```
... Top 10 Unigrams:

      term      score
91      data 0.111304
168    experience 0.093773
55      business 0.055090
396      sql 0.048216
418      teams 0.046887
82    collaboration 0.046887
85      creation 0.046887
387     solving 0.046887
88     dashboard 0.046887
65     candidate 0.046887
73    candidates 0.046887
76      cleaning 0.046887
436      years 0.046887
328     problem 0.046887
327    preferred 0.046887
355 responsibilities 0.046887
347    qualification 0.046887
352     reporting 0.046887
3      analysis 0.046887
263     looking 0.046887
```

6. Bigram Results

Bigrams were generated to identify important two-word combinations and potential multi-word skills.

```
-

Top 10 Bigrams:

      term      score
419    teams candidates 0.046887
388    solving collaboration 0.046887
83    collaboration business 0.046887
86      creation data 0.046887
77      cleaning problem 0.046887
60      business teams 0.046887
66    candidate experience 0.046887
74    candidates years 0.046887
329    problem solving 0.046887
356 responsibilities include 0.046887
353    reporting dashboard 0.046887
348    qualification preferred 0.046887
89      dashboard creation 0.046887
96      data cleaning 0.046887
92      data analysis 0.046887
4      analysis reporting 0.046887
207      include data 0.046887
437      years experience 0.046887
183    experience python 0.037805
7      analyst join 0.036272
```

7. Trigram Results

Trigrams were generated to identify important three-word combinations and potential multi-word skills.

Top 10 Trigrams:

	term	score
97	data cleaning problem	0.046887
5	analysis reporting dashboard	0.046887
420	teams candidates years	0.046887
78	cleaning problem solving	0.046887
84	collaboration business teams	0.046887
87	creation data cleaning	0.046887
389	solving collaboration business	0.046887
61	business teams candidates	0.046887
75	candidates years experience	0.046887
208	include data analysis	0.046887
330	problem solving collaboration	0.046887
357	responsibilities include data	0.046887
93	data analysis reporting	0.046887
354	reporting dashboard creation	0.046887
90	dashboard creation data	0.046887
69	candidate experience python	0.037805
112	degree qualification preferred	0.029719
426	technologies candidate experience	0.028426
344	python sql aws	0.027140
184	experience python pandas	0.025478

8. Multi-Word Skill Discovery

The N-Gram analysis helped identify multi-word terms that may represent technical skills, such as Machine Learning, Power BI, Data Science, and Natural Language Processing.

9. Result

The analysis generated Unigrams, Bigrams, and Trigrams from the job descriptions. The higher-scoring N-Grams were reviewed to identify potential multi-word skills.

Date: 29 AUG 2026

Day 11 – Word Embeddings and Semantic Similarity

1. Objective

The objective of Day 11 was to improve the skill extraction process using semantic similarity instead of relying only on exact keyword matching.

2. Problem with Exact Matching

Exact keyword matching may fail when two terms have the same meaning but are written differently.

For example:

ML

Machine Learning

Both terms refer to the same concept, but simple exact matching may not identify them as related.

3. Implementation

I used a skills dictionary and embedding- based approach to calculate the similarity between job-related txt and available skills.

The basic workflow is:

Skill Dictionary → Embeddings → Job Description → Similarity Score → Possible Skills

This approach helps identify skills that are semantically related even when the exact word is not present.

4. Process

1. Created a skill dictionary.
2. Converted the skills into embeddings.
3. Selected a job description.
4. Converted the job description into an embedding.
5. Calculated similarity scores between the job description and skills.
6. Sorted the skills based on similarity score.
7. Applied a similarity threshold to identify possible skills.

5. Result

The system generated similarity scores for the skills and identified possible skills based on the selected threshold.

Semantic Similarity Results:

	skill	similarity_score
0	Azure	0.348913
1	Hadoop	0.316351
2	AWS	0.306415
3	python	0.295567
4	SQL	0.271763
5	GCP	0.258586
6	Pandas	0.245894
7	Tableau	0.239481
8	Scikit-learn	0.235546
9	TensorFlow	0.218847

```
# Identify Potential Skill
threshold = 0.20
possible_skills = semantic_results[
    semantic_results["similarity_score"] >= threshold
]
possible_skills = possible_skills["skill"].tolist()
print("Possible Skills:", possible_skills)
```

Possible Skills: ['Azure', 'Hadoop', 'AWS', 'python', 'SQL', 'GCP', 'Panda

Date: 29 AUG 2026

DAY 12 – Named Entity Recognition

1. Objective

To understand Named Entity Recognition (NER) using spaCy and identify entities from job-related text.

2. Task Performed

I used the spaCy NLP library to identify named entities from job-related text.

Example: -

Python

Microsoft

AWS

Google Cloud

New York

```
#Standard spaCy NER
doc = nlp(text)
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

```
microsoft -> ORG
Google Cloude -> ORG
New york -> GPE
```

3.Implementation

I used the spaCy English NER model to process job-related text and identify entities.

The process is:

Job description → spaCy NER → Detected Entities → Custom Technical Entities

4. Process

1. Imported spaCy.
2. Loaded the English spaCy model.
3. Prepared job description text.
4. Applied spaCy NER.
5. Observed the entities detected by the standard NER model.
6. Checked whether technical skills were detected correctly.
7. Identified the limitation of standard NER.
8. Created custom entity categories for technical skills.
9. Used custom entities such as SKILL, TOOL, DATABASE, and CLOUD_PLATFORM

5. Custom Entities Created

For technical job descriptions, the following custom entity categories were considered:

Entity	Example
SKILL	Python
TOOL	Power BI
DATABASE	PostgreSQL
CLOUD-PLATFORM	AWS
TECHNOLOGY	Technical technology

```
#Apply custom entities
text = """
The candidate has experience in Python, SQL, Machine Learning,
Power BI, PostgreSQL, AWS and Google Cloud.
"""

for entity, label in custom_entities.items():
    if entity.lower() in text.lower():
        print(entity, "->", label)

Python -> SKILL
SQL -> SKILL
Machine Learning -> TECHNOLOGY
Power BI -> TOOL
PostgreSQL -> DATABASE
AWS -> CLOUD_PLATFORM
Google Cloud -> CLOUD_PLATFORM
```

Why Standard NER is Not Enough

Standard NER models are mainly trained to recognize common entity types such as people, organizations, locations, etc.

Technical skills such as Python, PostgreSQL or Power BI may not always be recognized correctly as named entities.

Therefore, custom entities are required for technical skill extraction. The task specifically asks to investigate why standard NER fails to recognize many technical skills.

The NER process helped understand how standard NLP models identify entities and why custom entity recognition is useful for technical job descriptions.

Date: - 31 AUG 2026

DAY: - 13 Build a Machine Learning Skill Classifier

1. Objective

The objective of Day 13 is to build a supervised machine learning model for classifying skills and technologies into predefined categories. Labeled training data is created by assigning appropriate categories to different technical entities. TF-IDF is used to convert text into numerical features, and Logistic Regression is used as the machine learning classifier. The complete pipeline follows the process of text preprocessing, TF-IDF vectorization, model training, and skill category prediction.

2. Task

In this task, labeled skill data was created and a Machine Learning classifier was developed to classify technical skills into appropriate categories.

Example: -

Text Entity	Label
Python	SKILL
AWS	CLOUD
SQL	DATABASE
Power BI	BI_TOOL

3. Labeled Training Data

The first step in building a supervised machine learning classifier is to create labeled training data. Each technical entity is associated with a corresponding category label. These labels provide the target information required by the machine learning model during training.

This labeled data enables the machine learning model to learn patterns between the input text and the assigned skill category.

The following examples demonstrate how technical entities can be categorized:

...	Text	Label
0	Python	SKILL
1	Pandas	SKILL
2	NumPy	SKILL
3	Machine Learning	SKILL
4	AWS	CLOUD
5	Azure	CLOUD
6	Google Cloud	CLOUD
7	SQL	DATABASE
8	PostgreSQL	DATABASE
9	MySQL	DATABASE
10	Power BI	BI_TOOL
11	Tableau	BI_TOOL
12	Excel	BI_TOOL

3. Approach Used

TF-IDF + Logistic Regression was used to build the supervised Machine Learning pipeline.

4. Data Preparation

Before training the machine learning model, the text data is prepared for processing. The input text is converted into a suitable format and unnecessary variations are handled so that the classifier can learn effectively.

The prepared text is then passed to the TF-IDF vectorizer. The dataset is divided into input features and target labels, where the text represents the input and the skill category represents the target variable.

Labeled training data was prepared by assigning predefined categories to different technical skills.

Example:

Python → SKILL

AWS → CLOUD

SQL → DATABASE

Power BI → BI_TOOL

```
#Check Training data

print("Dataset Shape:", training_data.shape)

print("\nLabel Distribution:")
print(training_data["Label"].value_counts())

print("\nMissing Values:")
print(training_data.isnull().sum())
```

... Dataset Shape: (13, 2)

Label Distribution:

Label

SKILL 4

CLOUD 3

DATABASE 3

BI_TOOL 3

Name: count, dtype: int64

Missing Values:

Text 0

Label 0

dtype: int64

5. TF-IDF Vectorization

TF-IDF (Term Frequency–Inverse Document Frequency) is used to convert textual data into numerical features that can be understood by a machine learning algorithm.

TF-IDF assigns importance to words based on their occurrence in the text while reducing the importance of terms that appear frequently across documents. This representation allows the Logistic Regression model to work with textual skill information.

In this project, TF-IDF is used as the feature extraction technique before passing the data to the machine learning classifier. TF-IDF was used to convert the skill text into numerical features that can be provided to the Machine Learning model.

Simple explanation:

Text → TF-IDF → Numerical Features

6. Machine Learning Model

Logistic Regression was used as the supervised classification model to learn the relationship between skill names and their corresponding categories.

A supervised machine learning classifier is used to classify the textual entities into predefined skill categories.

Several machine learning approaches can be used for text classification, including Logistic Regression, Random Forest, and Support Vector Machine (SVM). For this task, Logistic Regression is used together with TF-IDF as the main classification approach.

Logistic Regression is suitable for text classification because it can learn the relationship between numerical text features and categorical target labels.

Pipeline:

Text

↓

Preprocessing

↓

TF-IDF

↓

Logistic Regression

↓

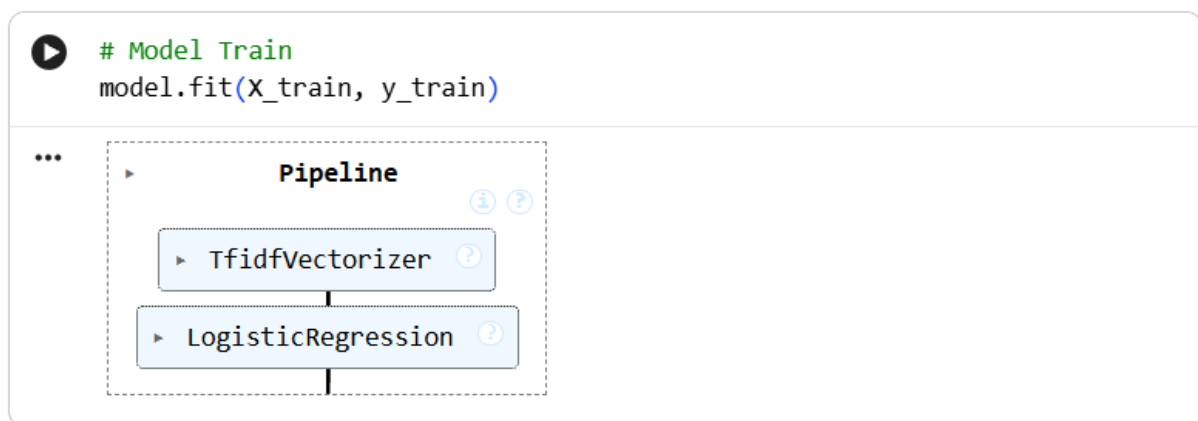
Skill Category

7. Model Training

The labeled skill dataset was divided into training and testing data. The TF-IDF features were used to train the Logistic Regression classifier.

The prepared training data is used to train the Logistic Regression classifier. The TF-IDF features are provided as input to the model, while the predefined skill categories are used as target labels.

During training, the model learns the relationship between textual patterns and their corresponding categories. After successful training, the model can be used to classify new skill-related text.



8. Prediction / Testing

After training the classifier, it is tested on text data that was not used during the training process. The trained model predicts the category of the given textual entity based on the patterns learned during training.

The prediction stage demonstrates whether the model can correctly classify new skill-related text into the predefined categories.

The diagram shows a Jupyter Notebook cell with a code snippet and a table of test results. The code snippet is:

```
# Test Predictions

y_pred = model.predict(X_test)

results = pd.DataFrame({
    "Text": X_test,
    "Actual_Label": y_test,
    "Predicted_Label": y_pred
})

results
```

Below the code, there is a table showing the results of the predictions. The table has four columns: "Text", "Actual_Label", and "Predicted_Label". The table contains four rows of data:

	Text	Actual_Label	Predicted_Label
0	Python	SKILL	SKILL
9	MySQL	DATABASE	SKILL
4	AWS	CLOUD	SKILL
12	Excel	BI_TOOL	SKILL

9. Results and Observations

The trained machine learning classifier was used to predict categories for skill-related text. The model was able to classify textual entities based on the patterns learned from the labeled training data.

The performance of the classifier is analyzed using the actual testing results obtained from the notebook. The observations help in understanding how effectively the model distinguishes between different skill categories.

Model Performance:

Accuracy: [0.25]

The final observations are based on the actual prediction and evaluation results obtained during model testing.

 **# Classification Report**

```
from sklearn.metrics import classification_report

print(classification_report(
    y_test,
    y_pred,
    zero_division=0
))
```

...	precision	recall	f1-score	support
BI_TOOL	0.00	0.00	0.00	1
CLOUD	0.00	0.00	0.00	1
DATABASE	0.00	0.00	0.00	1
SKILL	0.25	1.00	0.40	1
accuracy			0.25	4
macro avg	0.06	0.25	0.10	4
weighted avg	0.06	0.25	0.10	4

Date: - 13 Aug 2026

Day 14: - Advanced Transformer-Based Extraction

1.Objective

The objective of Day 14 is to explore advanced Transformer-based Natural Language Processing techniques for extracting skills and entities from job descriptions.

In this task, a pretrained Transformer-based Named Entity Recognition (NER) model is used as a baseline for identifying relevant entities from job descriptions. The extracted entities are then analyzed to identify skills and technologies relevant to the job domain.

This task helps in understanding the progression from traditional NLP techniques to modern Transformer-based approaches.

2. Transformer Models

Transformer-based NLP models use attention mechanisms to understand the relationship between words in a sentence.

BERT (Bidirectional Encoder Representations from Transformers):

BERT is a Transformer-based language model that learns contextual representations of words by considering information from both directions of the text.

DistilBERT:

DistilBERT is a smaller and faster version of BERT designed to retain much of BERT's language understanding capabilities while reducing computational requirements.

RoBERTa:

RoBERTa is an optimized version of BERT that uses improved training strategies to achieve better language understanding performance.

BERT-NER:

BERT-based NER models use Transformer architecture to identify and classify entities present in text.

Traditional NLP

↓

TF-IDF

↓

Word Embeddings

↓

NER

↓

Transformers

The project demonstrates the progression of Natural Language Processing techniques from traditional approaches to advanced Transformer-based methods.

Traditional NLP techniques mainly depend on predefined rules and keyword patterns. TF-IDF represents text numerically based on word importance. Word embeddings provide vector representations that capture relationships between words. NER identifies entities from text, while Transformer models provide stronger contextual understanding of language.

Transformer-based approaches therefore provide a more advanced foundation for extracting meaningful entities from unstructured job descriptions.

5. Transformer-Based NER Approach

For the practical implementation, a pretrained Transformer-based NER model was used as a baseline. The model used in this project is `dslim/bert-base-NER`.

The pretrained model processes the job description and identifies entities present in the text. The extracted entities are then examined and matched against the available skill vocabulary to retain relevant technical skills.

This approach allows the project to explore how a pretrained Transformer model can be applied to the skill extraction problem without training a Transformer model from scratch.

6. Input and Expected Output

The Transformer-based extraction process takes a job description as input and identifies relevant entities from the text.

Example Input:

"Experience with Python, SQL and AWS."

```
# Final Skill Output

final_output = pd.DataFrame({
    "Job Description": [text],
    "Extracted Skills": [", ".join(matched_skills)]
})

final_output
```

	Job Description	Extracted Skills
0	Experience with Python, SQL and AWS.	Python, SQL, AWS

This demonstrates the expected objective of identifying relevant skills from a job description.

7. Implementation Methodology

The Transformer-based skill extraction process consists of the following steps:

Step 1 – Input Job Description:

Job descriptions are provided as input text.

Step 2 – Transformer NER Processing:

The pretrained BERT-based NER model processes the input text and identifies entities.

Step 3 – Entity Extraction:

The entities identified by the model are extracted from the text.

Step 4 – Skill Filtering:

The extracted entities are compared with the available skill vocabulary to identify relevant skills.

Step 5 – Final Skill Prediction:

The matching entities are stored as the predicted skills for the job description.

Flow:

Job Description

↓

Pretrained BERT-NER Model

↓

Entity Detection

↓

Skill Vocabulary Matching

↓

Extracted Skills

```

# Skill extractor on each skill

all_results = []

for text in test_descriptions:

    matched_skills = []

    for skill in skill_dictionary:
        if skill.lower() in text.lower():
            matched_skills.append(skill)

    all_results.append({
        "Job Description": text,
        "Extracted Skills": ", ".join(matched_skills)
    })

transformer_skill_results = pd.DataFrame(all_results)

transformer_skill_results

```

	Job Description	Extracted skills
0	Experience with Python, SQL and AWS.	Python, SQL, AWS
1	Strong knowledge of Power BI, Excel and Tableau.	Power BI, Tableau
2	Experience with Machine Learning, Python and P...	Python, SQL, Machine Learning, PostgreSQL

Date: - 1 SEP 2026

DAY 15: - Model Evaluation

1. Objective

The objective of Day 15 was to evaluate and compare different skill extraction approaches used in the project. The models were evaluated based on their ability to correctly identify required skills from job descriptions.

2. Models Evaluated

The following approaches were considered for comparison:

Dictionary Matching

Regex

TF-IDF

NER

Machine Learning Model

Transformer-based Model

3. Evaluation Metrics

The models were evaluated using Precision, Recall, F1-Score, and Accuracy.

Precision – Measures how many predicted skills were actually correct.

Recall – Measures how many actual skills were successfully identified.

F1-Score – Provides a balance between Precision and Recall.

Accuracy – Measures the overall correctness of predictions.

For skill/entity extraction, Precision, Recall and F1-Score are more important than Accuracy, as mentioned in the task requirements.

... Precision, Recall and F1 Score calculated successfully!

	job_title	TP	FP	FN	Precision	Recall	F1_Score
0	BI Analyst	4	0	0	1.0	1.0	1.0
1	ML Engineer	4	0	0	1.0	1.0	1.0
2	Financial Analyst	4	0	0	1.0	1.0	1.0
3	Data Analyst	4	0	0	1.0	1.0	1.0
4	product analyst	4	0	0	1.0	1.0	1.0
5	Software Engineer	4	0	0	1.0	1.0	1.0
6	Financial Analyst	4	0	0	1.0	1.0	1.0
7	Data Scientist	4	0	0	1.0	1.0	1.0
8	Product Analyst	4	0	0	1.0	1.0	1.0
9	MIS Executive	4	0	0	1.0	1.0	1.0

4. Model Comparison

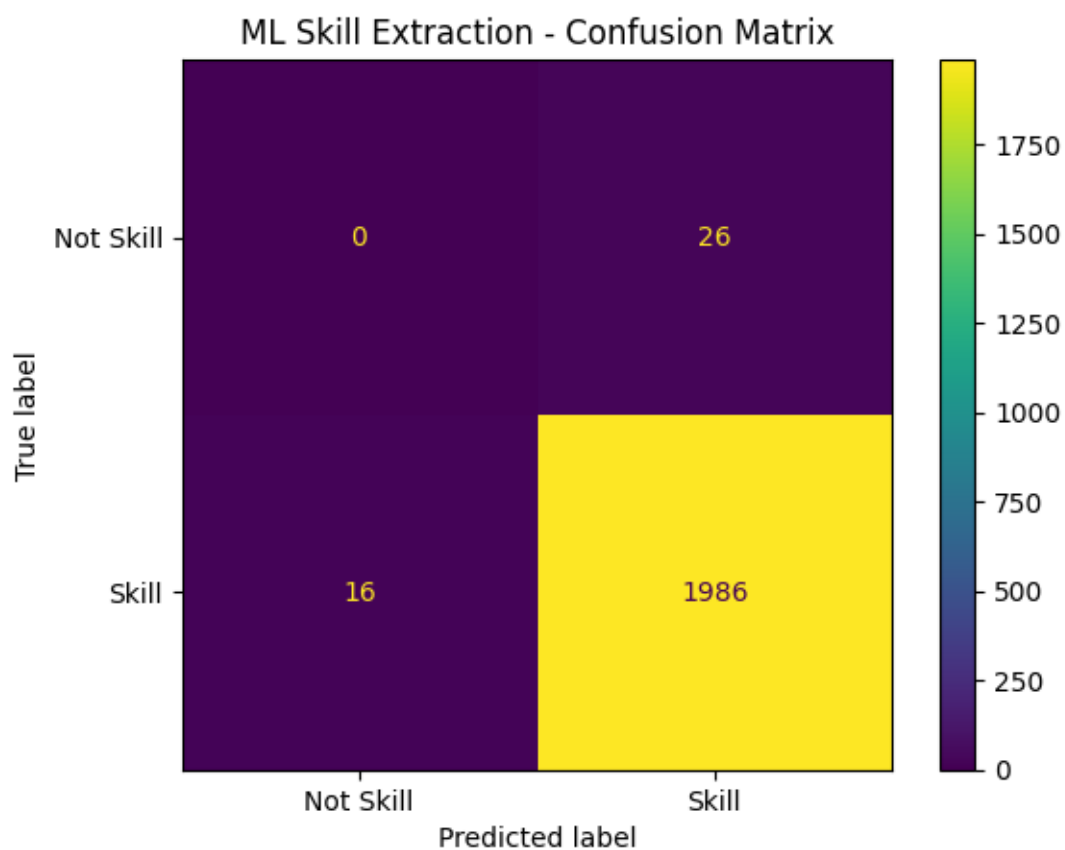
The performance of the different approaches was compared using their calculated evaluation metrics. This comparison helped identify which approach performed better for extracting skills from job descriptions.

	Method	Accuracy	Precision	Recall	F1_Score
0	Dictionary	0.9529	0.9595	0.9782	0.9678
1	Regex	0.9870	0.9782	0.9782	0.9782
2	TF-IDF	0.9253	0.9589	0.9498	0.9516
3	NER	0.7316	0.9782	0.7317	0.8265
4	ML	0.9793	0.9861	0.9832	0.9842
5	Transformer	0.7207	0.9591	0.7339	0.8118

5. Confusion Matrix Analysis

Confusion matrices were used to analyze the prediction results by identifying correct and incorrect classifications. They helped understand the model's prediction performance in more detail.

ML Accuracy: 0.9793



6. Results and Observations

The evaluation showed that different approaches have different strengths and limitations. Traditional methods such as dictionary and regex matching are useful for exact skill identification, while ML and Transformer-based approaches can provide more advanced extraction capabilities.

7. Overall Evaluation

The models were compared using the calculated Precision, Recall, F1-Score and Accuracy values. The evaluation results provide a basis for selecting the most suitable approach for skill extraction from job descriptions.

Date: - 1 SEP 2026

DAY 16: - Error Analysis

1. Objective

The objective of Day 16 was to identify and analyze cases where the skill extraction model failed to correctly extract skills from job descriptions. Error analysis helps understand the limitations and improve the model.

2. Error Analysis

The actual and predicted skills were compared for each job description. The differences between the expected skills and extracted skills were analyzed to identify different types of prediction errors.

3. Error Categories

The task required analysis of the following error categories:

- False Positive
- False Negative
- Wrong Entity
- Partial Entity
- Duplicate Skill
- Unknown Skill
- Synonym Problem
- Spelling Problem

These categories help identify different reasons for incorrect skill extraction.

4. Detected Errors

In the performed analysis, the following major error categories were identified:

Error Category	Count
False Negative	2598
False Positive	275
Total Error rows	2873

The results show that False Negative was the major error category, indicating that some actual skills were not successfully extracted by the model.



Step 64: Check Error Categories

```
print("Error categories found:")
print(error_analysis_df["Error"].value_counts())

print("\nTotal error rows:", len(error_analysis_df))
```

```
... Error categories found:
Error
False Negative    2598
False Positive     275
Name: count, dtype: int64
```

```
Total error rows: 2873
```

5. Error Analysis Table

A detailed error table was created containing the Text, Actual, Predicted and Error columns. This table provides individual examples of incorrect or incomplete skill extraction.

DAY 16 FINAL VERIFICATION

=====

```
Excel file exists: True
Columns: ['Text', 'Actual', 'Predicted', 'Error']
Total error rows: 2988
Required columns present: True
```

```
Final Error Categories:
Error
False Negative    2455
False Positive     268
Wrong Entity       265
Name: count, dtype: int64
```

First 5 Error Cases:

	Text	Actual	Predicted	Error
0	We are looking for a BI Analyst to join Insigh...	sql		False Negative
1	We are looking for a ML Engineer to join BlueP...	rest api		False Negative
2	We are looking for a Financial Analyst to join...	dax		False Negative
3	We are looking for a Financial Analyst to join...	excel		False Negative
4	We are looking for a Financial Analyst to join...	power bi		False Negative

6. False Negative and False Positive Analysis

False Negative errors occurred when an actual skill was present but was not extracted by the model. False Positive errors occurred when the model predicted a skill that was not part of the expected skill list.

7. Wrong Entity Analysis

Wrong Entity errors were identified when the extracted entity did not correctly match the expected skill. These errors show that the model may sometimes identify an incorrect entity or skill representation from the job description.

```
... Spelling Problem cases: 0
Wrong Entity cases: 273
```

```
Sample Spelling Errors:
```

```
—
```

```
Sample Wrong Entity Errors:
```

	Text	Actual	Predicted	Error
0	We are looking for a Power BI Developer to joi...	sql	power bi	Wrong Entity
1	We are looking for a Data Scientist to join Da...	nan	etl	Wrong Entity
2	We are looking for a Data Scientist to join Da...	nan	python	Wrong Entity
3	We are looking for a Data Scientist to join Da...	nan	sql	Wrong Entity
4	We are looking for a Data Scientist to join Da...	nan	aws	Wrong Entity
5	We are looking for a Python Developer to join ...	rest api	python	Wrong Entity
6	We are looking for a Power BI Developer to joi...	rest api	power bi	Wrong Entity
7	We are looking for a Power BI Developer to joi...	spring boot	power bi	Wrong Entity
8	We are looking for a Python Developer to join ...	excel	python	Wrong Entity

Date: - 2 SEP 2026

Day 17: - Skill Normalization

1. Objective

The objective of Day 17 was to build a skill normalization process that converts different variations of the same skill into a single canonical skill name. This improves consistency in the extracted skill data and makes the results suitable for further analysis and visualization.

2. Existing Skill Taxonomy and Aliases

The skill taxonomy and aliases created during Day 5 were reused for the normalization process. Existing mappings such as python3 → Python, postgres → PostgreSQL, and powerbi → Power BI were used as the basis for standardizing skill names.

```
# Check existing variables related to aliases / taxonomy

[x for x in globals().keys() if "alias" in x.lower() or "taxonom" in x.lo

['taxonomy_df',
 'aliases',
 'taxonomy_data',
 'skill_taxonomy_df',
 'skill_taxonomy']
```

3. Normalization Process

A normalization engine was implemented using the following process:

Raw Skill → Lowercase → Remove Special Characters → Alias Matching → Canonical Skill

This process ensures that different representations of the same technology are converted into a consistent skill name.

```
import re

def normalize_skill(raw_skill):
    if raw_skill is None:
        return None
    # Convert to lowercase and remove extra spaces
    skill = str(raw_skill).strip().lower()
    skill = re.sub(r"\s+", " ", skill)
    # Check alias BEFORE removing special characters
    if skill in aliases:
        return aliases[skill]

    # Remove special characters
    skill_clean = re.sub(r"[^a-z0-9\s]", "", skill)
    skill_clean = re.sub(r"\s+", " ", skill_clean).strip()

    # Check alias again after cleaning
    if skill_clean in aliases:
        return aliases[skill_clean]

    # Handle common Python variations
    if skill_clean in ["python3", "python 3", "python programming"]:
        return "Python"

    return skill_clean.title()

print("Skill normalization function updated successfully!")

... Skill normalization function updated successfully!
```

4. Skill Normalization Testing

The normalization logic was tested with different skill variations such as Python3, Python 3, PowerBI, Power-BI, Postgres, and PostgreSQL. The test results were checked to verify that skill variations were mapped to their appropriate canonical forms.

...

	Raw Skill	Normalized Skill
0	Python3	Python
1	Python 3	Python
2	Python programming	Python
3	PowerBI	Power BI
4	Power BI	Power BI
5	Power-BI	Power BI
6	postgres	PostgreSQL
7	PostgreSQL	PostgreSQL

5. Applying Normalization to the Dataset

The normalization engine was applied to the job dataset using the existing keywords field. Each job's skills were processed and converted into normalized skill lists while preserving the associated job role.

... Actual dataset skills normalized successfully!
Total unique raw skills: 20

	Raw Skill	Normalized Skill
0	Tableau	Tableau
1	SQL	Sql
2	Excel	Excel
3	Business Intelligence	Business Intelligence
4	Java	Java
5	Spring Boot	Spring Boot
6	REST API	Rest Api
7	Power BI	Power BI
8	DAX	Dax
9	Data Visualization	Data Visualization
10	Python	Python
11	AWS	Aws
12	ETL	Etl
13	Pandas	Pandas
14	NumPy	Numpy
15	Machine Learning	Machine Learning

6. Normalized Skill Dataset

After applying the normalization process, a structured normalized skill dataset was created containing the job role, original keywords, and normalized skills. This output provides a consistent representation of skills for subsequent analytics and dashboard development.

...	job_title	keywords	Normalized_Skills
0	BI Analyst	Tableau, SQL, Excel, Business Intelligence	[Tableau, Sql, Excel, Business Intelligence]
1	ML Engineer	Java, Spring Boot, SQL, REST API	[Java, Spring Boot, Sql, Rest Api]
2	Financial Analyst	SQL, Power BI, Excel, DAX	[Sql, Power Bi, Excel, Dax]
3	Data Analyst	Excel, Power BI, SQL, Data Visualization	[Excel, Power Bi, Sql, Data Visualization]
4	product analyst	SQL, Power BI, Excel, DAX	[Sql, Power Bi, Excel, Dax]
5	Software Engineer	SQL, Power BI, Excel, DAX	[Sql, Power Bi, Excel, Dax]
6	Financial Analyst	Python, SQL, AWS, ETL	[Python, Sql, Aws, Etl]
7	Data Scientist	Python, Pandas, NumPy, SQL	[Python, Pandas, Numpy, Sql]
8	Product Analyst	Python, Pandas, Machine Learning, Scikit-learn	[Python, Pandas, Machine Learning, Scikitlearn]
9	MIS Executive	Python, SQL, AWS, ETL	[Python, Sql, Aws, Etl]

Date: - 2 SEP 2026

DAY 18: - BUSINESS DASHBOARD REPORT

1. Objective

The objective of Day 18 was to convert the NLP-based job skill extraction results into an interactive business analytics dashboard using Microsoft Power BI. The dashboard provides insights into skill demand, job roles, and skill categories.

2. Data Used

The Power BI dashboard was created using the processed Job Skill Analytics dataset containing job roles, extracted skills, and skill categories.

The main fields used for dashboard analysis were:

Job_Role

Skill

Skill_Category

Job_ID

3. Dashboard KPIs

Six key performance indicators were created to provide a quick summary of the dataset:

Total Jobs – Displays the number of unique jobs.

Unique Skills – Displays the total number of unique skills.

Top Skill – Shows the most frequently required skill.

Top Job Role – Shows the job role with the highest skill demand.

Most Required Technology – Highlights the most frequently required technology/skill.

Top Skill Category – Shows the most demanded skill category.

4. Dashboard Visualizations

The following visualizations were created:

a. Top 20 Most Required Skills

A bar chart was created to display the top 20 skills based on their frequency in job requirements.

b. Skill Demand by Job Role

A stacked bar chart was used to analyze the skills required across different job roles.

c. Skill Category Distribution

A donut chart was created to show the distribution of skills across categories such as Programming, Database, Cloud, Analytics, and Machine Learning.

5. Interactive Slicers

Two slicers were added to make the dashboard interactive:

Job Role – Allows users to filter the dashboard based on a specific job role.

Skill Category – Allows users to analyze skills belonging to a particular category.

The slicers dynamically update the dashboard KPIs and visualizations.

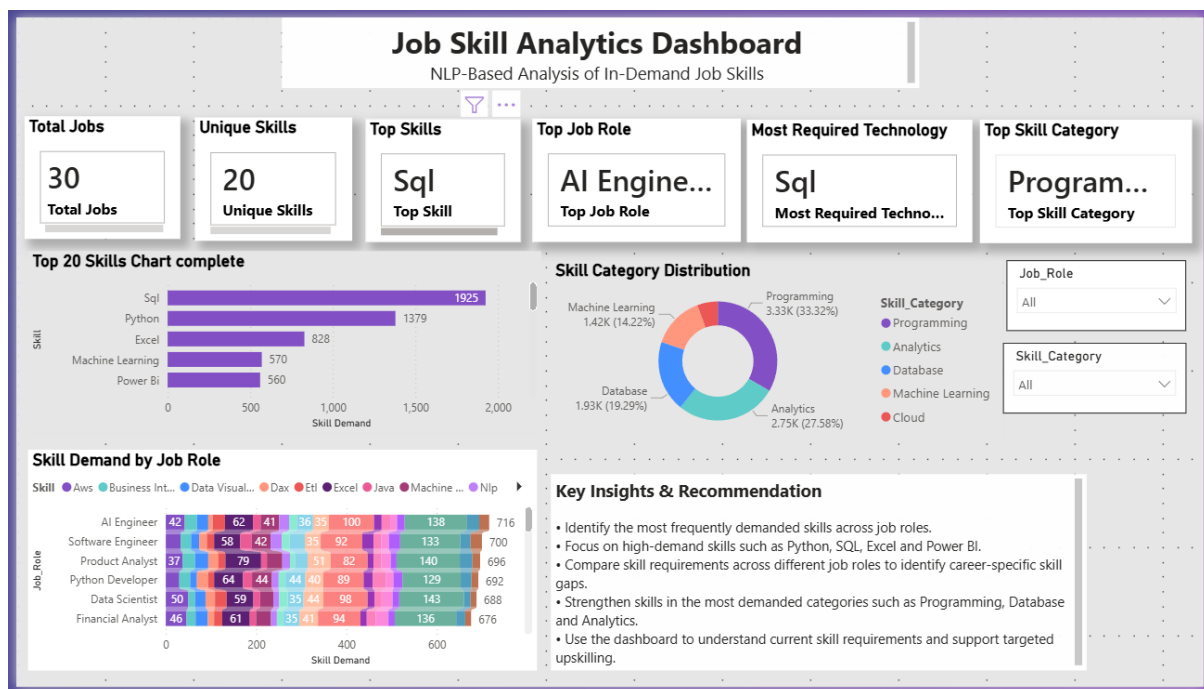
6. Skill Trends

Skill Trends by month/year was not implemented because the dataset does not contain a date, month, or year field. Therefore, no artificial date information was added.

7. Dashboard Insights

The dashboard helps identify the most frequently demanded skills, compare skill requirements across job roles, and understand the distribution of skills across different categories.

These insights can help job seekers identify high-demand technical skills and understand the skill requirements associated with different job roles.



Date: - 4 SEP 2026

DAY 19: - Build an Application/API

1. Objective

The objective of Day 19 was to convert the Job Skill Extraction NLP project into a usable web application. The application allows users to enter a job description and automatically detects technical skills, normalizes them, and maps them to relevant skill categories.

2. Application Overview

SkillExtract AI is a Python and Streamlit-based web application developed to extract technical skills from job descriptions. It provides a simple interface where users can paste a job description and get the detected skills along with their categories.

3. Technologies Used

- Python
- Streamlit
- Pandas
- NLP-based skill extraction
- Skill Taxonomy
- Skill Normalization
- Category Mapping

5. Application Architecture

User

↓

Login / Sign up

↓

Streamlit Web Interface

↓

Job Description

↓

Skill Extraction

↓

Skill Normalization

↓

Category Mapping

↓

Final Skills & Categories

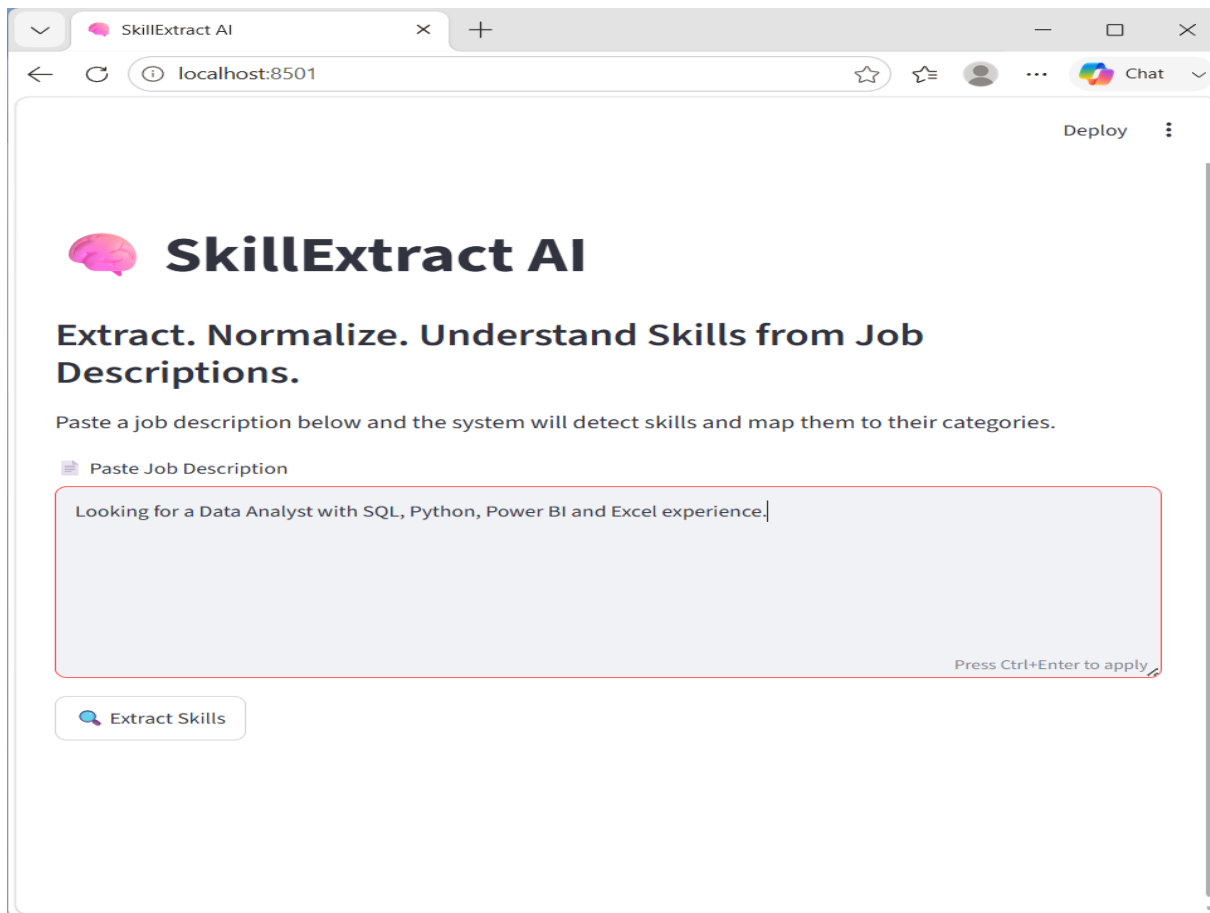
6. Features

- User registration and login
- Personalized welcome message after login
- Paste Job Description
- Detect technical skills
- Normalize detected skills
- Categorize skills
- Display results in a simple web interface
- Handles empty input
- Can process different job descriptions
- Logout functionality

7. Application Input

Looking for a Data Analyst with SQL, Python, Power BI and Excel experience.

Screenshot 1: Job description input in SkillExtract AI



The screenshot shows a web browser window with the title 'SkillExtract AI'. The address bar shows 'localhost:8501'. The page has a 'Deploy' button in the top right corner. The main heading is 'SkillExtract AI' with a brain icon. Below it is the tagline 'Extract. Normalize. Understand Skills from Job Descriptions.' and a prompt 'Paste a job description below and the system will detect skills and map them to their categories.' There is a 'Paste Job Description' button. A large text input area contains the text 'Looking for a Data Analyst with SQL, Python, Power BI and Excel experience.' and a hint 'Press Ctrl+Enter to apply'. At the bottom is an 'Extract Skills' button.

SkillExtract AI

Extract. Normalize. Understand Skills from Job Descriptions.

Paste a job description below and the system will detect skills and map them to their categories.

Paste Job Description

Looking for a Data Analyst with SQL, Python, Power BI and Excel experience.

Press Ctrl+Enter to apply

Extract Skills

8. Application Output

Expected output:

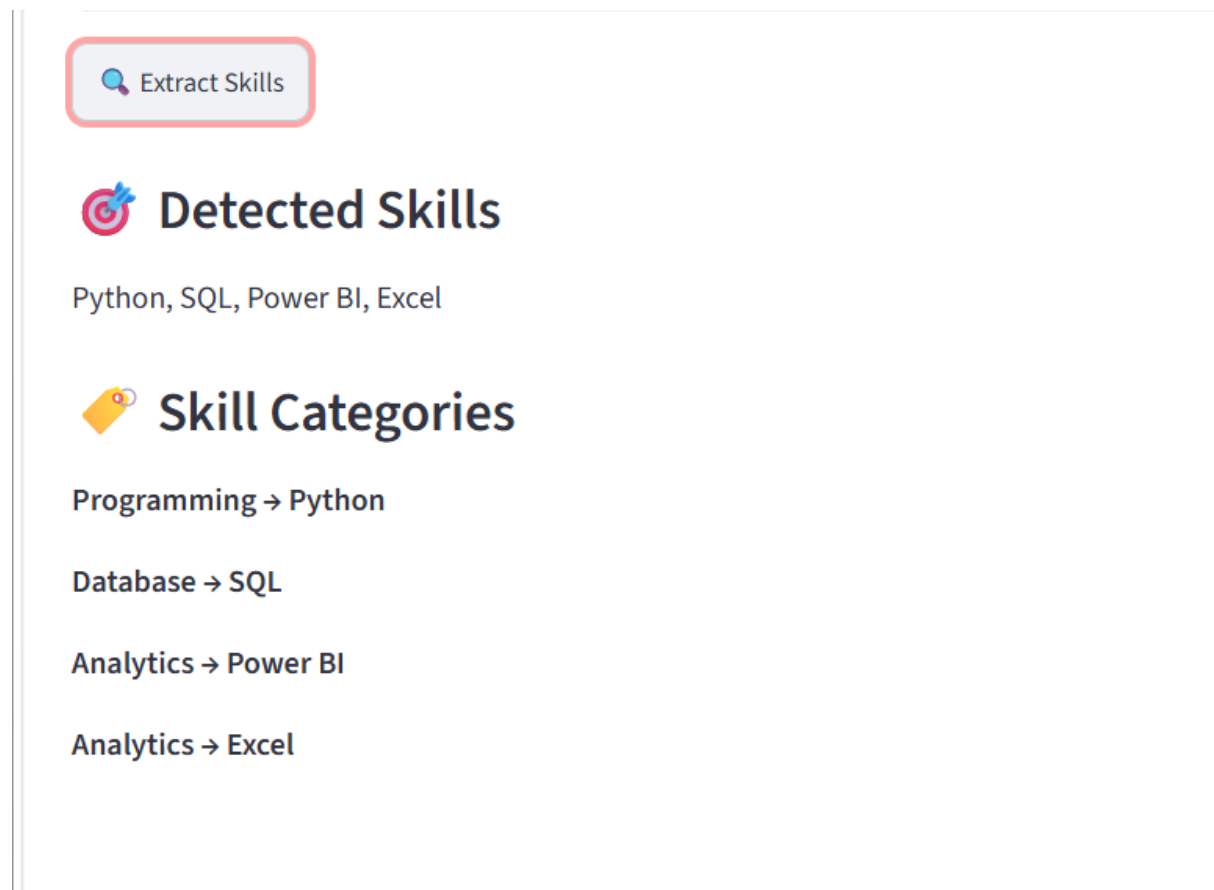
Detected Skills

- Python
- SQL
- Power BI
- Excel

Categories:

- Programming → Python
- Database → SQL
- Analytics → Power BI
- Analytics → Excel

Screenshot 2: Extracted skills and skill Categories.



9. Testing

The SkillExtract AI application was tested using different job descriptions to verify whether the system correctly detects technical skills and maps them to the appropriate categories.

Input Screenshot: - Testing SkillExtract AI with a different job description

The screenshot shows a web browser window with the URL `localhost:8501`. The page has a header with a "Deploy" button. The main content area is titled "Paste Job Description" and contains a text box with the following text: "We are looking for a Data Scientist with experience in Python, Pandas, NumPy, Scikit-learn and AWS." Below the text box is a button labeled "Extract Skills". Underneath the button, the page displays "Detected Skills" as "Python, AWS, Pandas, NumPy, Scikit-learn". Below this, it shows "Skill Categories" with a list of mappings: "Programming → Python", "Cloud → AWS", "Programming → Pandas", "Programming → NumPy", and "Machine Learning → Scikit-learn".

The application successfully detected the relevant skills and mapped them to their respective categories.

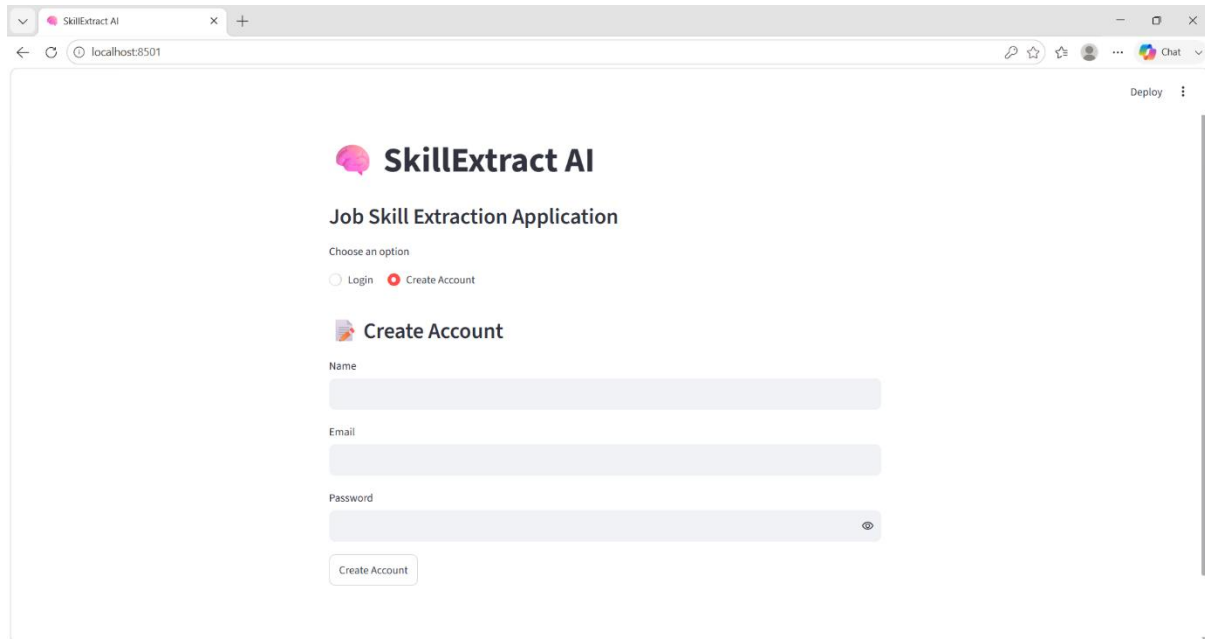
10.Login / signup

The application includes a simple user authentication system. New users can create an account using their name, email and password. Registered users can log in and receive a personalized welcome message. A logout option is also provided for secure session management.

Login Page: -

The screenshot shows the login page of the SkillExtract AI application. The page has a header with a "Deploy" button. The main content area is titled "SkillExtract AI" and "Job Skill Extraction Application". Below the title, there is a section "Choose an option" with two radio buttons: "Login" (selected) and "Create Account". Below this is a "Login" section with a lock icon. It contains two input fields: "Email" and "Password". The "Password" field has a toggle icon to show or hide the password. Below the input fields is a "Login" button.

Create Account: -



SkillExtract AI

Job Skill Extraction Application

Choose an option

☐ Login ☒ Create Account

Create Account

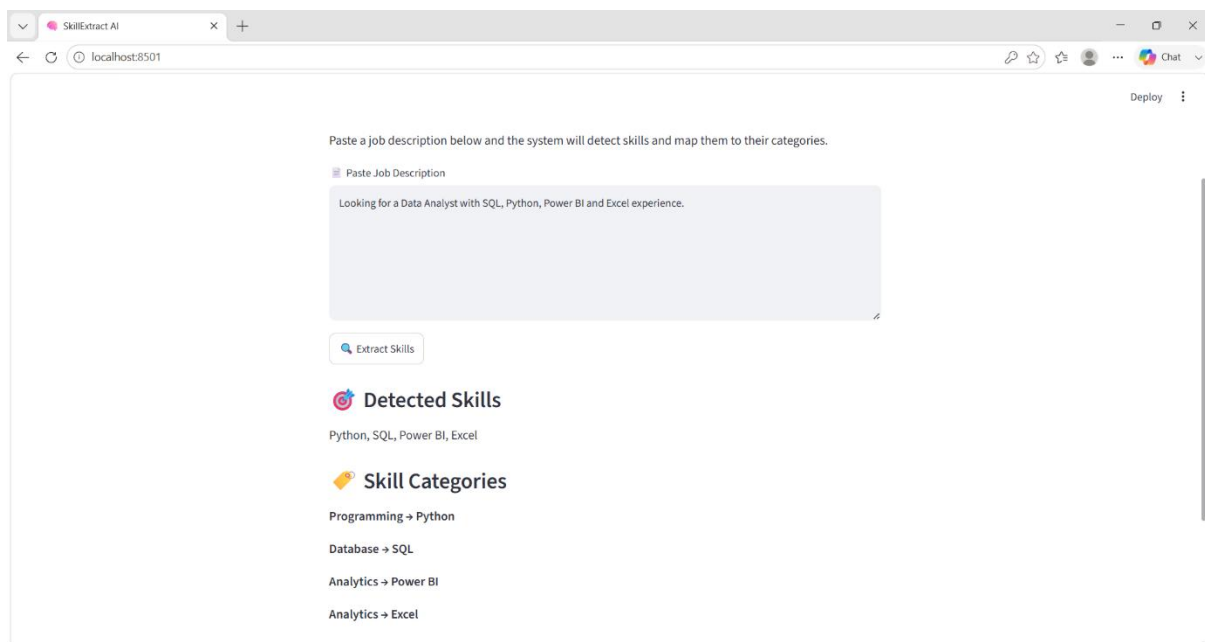
Name

Email

Password

Create Account

Final Output: -



Paste a job description below and the system will detect skills and map them to their categories.

Paste Job Description

Looking for a Data Analyst with SQL, Python, Power BI and Excel experience.

Extract Skills

Detected Skills

Python, SQL, Power BI, Excel

Skill Categories

Programming → Python

Database → SQL

Analytics → Power BI

Analytics → Excel

10. Result

The SkillExtract AI application was successfully developed and tested locally using Streamlit. It successfully accepts new job descriptions and identifies available technical skills and their corresponding categories.